

Duy Nguyen

Seattle, WA • dnguyen44@seattleu.edu • linkedin.com/in/duwe-ng • github.com/dcnguyen060899

Summary

Data scientist and MS candidate who designs and runs rigorous ML experiments—and builds the data pipelines they run on. Recent research isolated the retrieval *encoder*, not costly decoder fine-tuning, as the performance lever in medical-imaging RAG; the result is a first-author paper under review at PSB 2027. I own problems end to end: experiment design, modeling, evaluation, and reporting. Of those four, reporting has been the hardest to learn — it is where months of shared work have to be revised into a single coherent argument, and where a deadline leaves the least room to get it wrong.

Education

M.S. in Data Science — Seattle University

Expected Jun 2027

Seattle, WA

Certificate, Machine Learning & AI — UC Berkeley (Online)

Jul 2024

B.A. in Economics, Data Analysis Concentration — Simon Fraser University

May 2023

Burnaby, BC, Canada

Experience

Graduate Research Assistant — Medical Vision–Language RAG

Winter 2025 – Present

Seattle University | Advisor: Dr. Wenjing Yang

- Replicated and audited a published mammography-RAG benchmark on VinDr-Mammo (20,000 images / 5,000 patients) in **Python** and **PyTorch**, uncovering a metric-labeling artifact (weighted scores published as “macro”) that corrected how the reference numbers must be interpreted and compared.
- Designed and ran a 24-cell controlled experiment (4 encoder substrates × frozen/fine-tuned × 3 vision–language models) with **HuggingFace Transformers**, **ChromaDB**, and **scikit-learn** to isolate the causal driver of generated-report quality.
- Isolated the retrieval *encoder* as the lever: replacing the frozen OpenCLIP retriever with a domain-native, supervised-contrastively fine-tuned Mammo-CLIP encoder—decoder frozen throughout—raised suspicion macro-F1 0.35 to 0.48 (+**0.13**) and BI-RADS (+0.10) on LLaVA-Med; the fine-tuning step alone accounts for +0.11 (Qwen2.5-VL) and +0.06 (LLaVA-Med), with retrieval-insensitive MedGemma as a negative control.
- Automated evaluation with reproducible, self-verifying generators (**Python/pandas**) that recompute 100+ reported metrics byte-exact from raw model predictions; authored a 37-page report, condensed into a first-author manuscript submitted to the Pacific Symposium on Biocomputing (PSB) 2027, under review.

Research Data Engineer (Full-time, Summer)

Spring 2026 – Jul 2026

Computational Neuroscience Lab (Prof. Brian Fischer), Seattle University

- Supported an NIH CRCNS-funded computational neuroscience project (NIH-CRCNS-Fischer) on barn owl sound localization by building data infrastructure for neural recordings.
- Engineered an end-to-end **ETL** pipeline in **Python** (**pandas**, **SciPy**, **h5py**) converting 5+ proprietary electrophysiology formats (XDPHYS `.iid/.itd/.bf`, MATLAB `.mat`, headerless CSV) into a normalized **SQLite** database of 195 neurons and 1,325 recording passes, with a second researcher’s spike-reliability dataset staged and schema-provisioned for load.
- Designed the relational schema and ER model (documented in **LaTeX**) with foreign-key constraints and explicit data-quality rules (missing-data handling, tracked exclusions, path remapping), consolidating scattered lab files into one queryable source of truth.
- Tuned queries and ETL to meet <200 ms query and <30 s full-load targets, letting lab students self-serve neuron-level queries that previously required manual file handling.
- Ran **SQL/SQLite** schema and data-integrity audits that caught and corrected issues (all-zero rows, weak-tuning exclusions) before downstream analysis.

- “Will AI Make Human Work Worthless—or Priceless?”: a nine-act interactive data story (**D3.js**, **Scrollama.js**) with scroll-driven animation and a reader-controlled parameter widget, translating an original nested-CES general-equilibrium model of AI and cognitive labor into a narrative for non-specialists. Judged on narrative, data interpretation, scrollytelling, and visual design.

Skills

Programming: Python, SQL, JavaScript

ML / AI: PyTorch, HuggingFace Transformers, scikit-learn, Retrieval-Augmented Generation (RAG), vision-language models, QLoRA / parameter-efficient fine-tuning

Data: pandas, NumPy, SciPy, SQLite, ETL design, relational schema / ER design, data quality

Tools: Jupyter, Git, LaTeX, D3.js, Scrollama.js, Google Colab (A100 GPU), Ollama